ESP-IDF

Firmware Programming in Embedded Systems

ESP32-S3 Development with ESP-IDF

A practical guide to development environments, peripherals, networking, storage, real-time execution, and robust firmware design.



Author

PhD. Luciano Radrigan Figueroa

Professor, Universidad de Concepción

Included guided labs

ESP32-S3 basics

Peripheral interfaces, networking, MQTT, microSD, and real-time firmware.

About the author

Luciano Radrigan Figueroa, PhD., is a professor at the Universidad de Concepción and a postdoctoral researcher specializing in embedded systems, Industrial Internet of Things, and predictive maintenance. He holds B.Sc., M.Sc., and PhD degrees in Electronics and Electrical Engineering from the Universidad de Concepción.

His work combines wireless sensors, embedded hardware, signal processing, TinyML, deep learning, federated learning, and transfer learning to detect operational risks and predict failures in critical industrial machinery. This guide brings that hardware-to-deployment perspective into embedded firmware education.

Contents

1 Introduction to ESP-IDF **4**

2 ESP32-S3 Hardware Overview and Specifications **4**

3 Installing ESP-IDF on Windows, macOS, and Linux **6**

3.1 Common prerequisites . . . 6

3.2 Windows installation . . . 6

3.3 macOS installation . . . 7

3.4 Linux installation . . . 8

4 Visual Studio Code and the ESP-IDF Extension **9**

4.1 Ubuntu: Recommended Classroom Setup for Reliable VS Code Detection . . . 10

4.1.1 Step A: Check how VS Code is installed . . . 10

4.1.2 Step B: Install EIM from Espressif's APT repository . . . 10

4.1.3 Step C: Verify the installation before opening VS Code . . . 11

4.1.4 Step D: Configure the extension in VS Code . . . 11

4.1.5 Step E: Ubuntu workaround if the installation is visible to EIM but not selected by VS Code 12

4.1.6 Step F: Serial and debug permissions on Ubuntu . . . 12

4.1.7 Classroom acceptance test . . . 12

4.2 Troubleshooting: VS Code Cannot Detect an Existing ESP-IDF Installation . . . 13

4.2.1 Step 1: Verify ESP-IDF outside Visual Studio Code . . . 13

4.2.2 Step 2: Ask the extension to select the installed version . . . 13

4.2.3 Step 3: If the list is empty, check the EIM metadata file . . . 13

4.2.4 Step 4: Manual configuration when EIM detection still fails . . . 14

4.2.5 Step 5: Check User Settings versus Workspace Settings . . . 14

4.2.6 Step 6: WSL, SSH, containers, and remote VS Code . . . 15

4.2.7 Quick recovery checklist . . . 15

5 ESP-IDF Project Structure and Development Workflow **15**

6 GPIO, Pin Assignment, Remapping, and Conflicts **16**

6.1 What is a GPIO? . . . 17

6.2 Example classroom pin plan . . . 18

6.3 Changing I2C pins . . . 19

6.4 Typical conflicts to recognize . . . 20

7 Serial Communication Interfaces: UART, I2C, and SPI **21**

7.1 UART . . . 21
7.2 I2C . . . 22
7.3 SPI . . . 23

8 ADC Fundamentals and Example 23

9 Interrupts, Watchdogs, Dual-Core Programming, and Blocking Functions 24
9.1 Interrupts: reacting to asynchronous events . . . 25
9.2 Watchdog timers . . . 26
9.3 Programming both ESP32-S3 cores . . . 26
9.4 Blocking functions and responsive firmware . . . 28

10 Wi-Fi Station and TCP Client 28
10.1 Wi-Fi station initialization . . . 29
10.2 TCP client example . . . 29

11 MQTT Publish/Subscribe Example 30

12 microSD Storage Example 31

13 Connection Diagrams for Common Modules 32
13.1 I2C sensor: BME280 / MPU6050 . . . 32
13.2 UART sensor: GPS module . . . 33
13.3 SPI storage: microSD module . . . 34

14 Complete Hello World Build, Flash, and Monitor Example 34
14.1 Step 1 - Open the example . . . 35
14.2 Step 2 - Select the target . . . 35
14.3 Step 3 - Optional project configuration . . . 35
14.4 Step 4 - Build the firmware . . . 36
14.5 Step 5 - Identify the serial port . . . 36
14.6 Step 6 - Flash the firmware . . . 36
14.7 Step 7 - Monitor the program . . . 37
14.8 Minimal custom Hello World . . . 37

15 Troubleshooting and Best Practices 37

16 Recommended References 38

1 Introduction to ESP-IDF

ESP-IDF (Espressif IoT Development Framework) is Espressif's official development framework for the ESP32 family. It includes the compiler toolchain, CMake/Ninja build system, FreeRTOS integration, peripheral drivers, networking stacks, security libraries, flashing tools, logging, debugging support, and a large collection of reference examples.

For students, the most important idea is that ESP-IDF is not just a library. It is a complete embedded-software environment. A typical workflow moves from source code to project configuration, compilation, binary generation, flashing, and finally runtime monitoring on the target board.

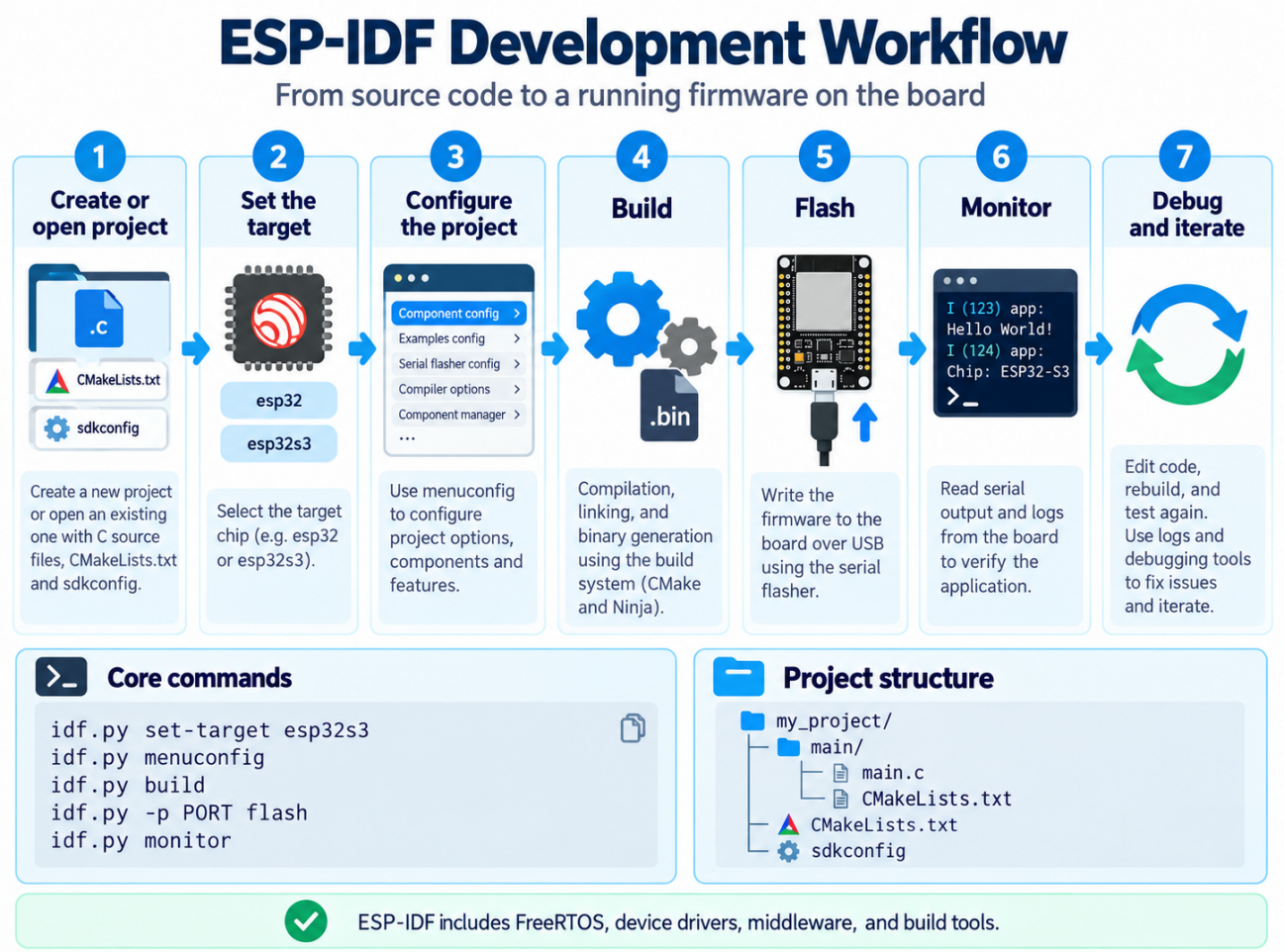


Figure 1. Typical ESP-IDF development workflow.

Course recommendation: Use one fixed stable ESP-IDF version for the whole class. This reduces differences between computers and keeps screenshots, APIs, examples, and troubleshooting steps consistent.

2 ESP32-S3 Hardware Overview and Specifications

The ESP32-S3 is a low-power system-on-chip designed for connected embedded systems. It combines two high-performance Xtensa LX7 CPU cores, Wi-Fi, Bluetooth Low Energy, extensive GPIO routing, analog peripherals, USB, hardware security, and multiple serial interfaces.

Feature	ESP32-S3 summary
CPU	Dual-core 32-bit Xtensa LX7, up to 240 MHz
Wireless	2.4 GHz Wi-Fi (IEEE 802.11 b/g/n) and Bluetooth 5 LE
Internal memory	384 KB ROM, 512 KB SRAM, 16 KB RTC SRAM
GPIO	45 physical GPIOs at chip level; board exposure depends on module/dev board
ADC	Two 12-bit SAR ADCs, up to 20 channels
UART	3 UART controllers
I2C	2 I2C controllers
SPI	General-purpose SPI plus SPI used for flash/PSRAM
PWM	LEDC (up to 8 channels) and MCPWM
USB	Full-speed USB OTG and USB Serial/JTAG
Other peripherals	I2S, SD/MMC, TWAI (CAN), RMT, pulse counter, timers, touch, temperature sensor
Security	Secure boot, flash encryption, AES/SHA/RSA/HMAC accelerators

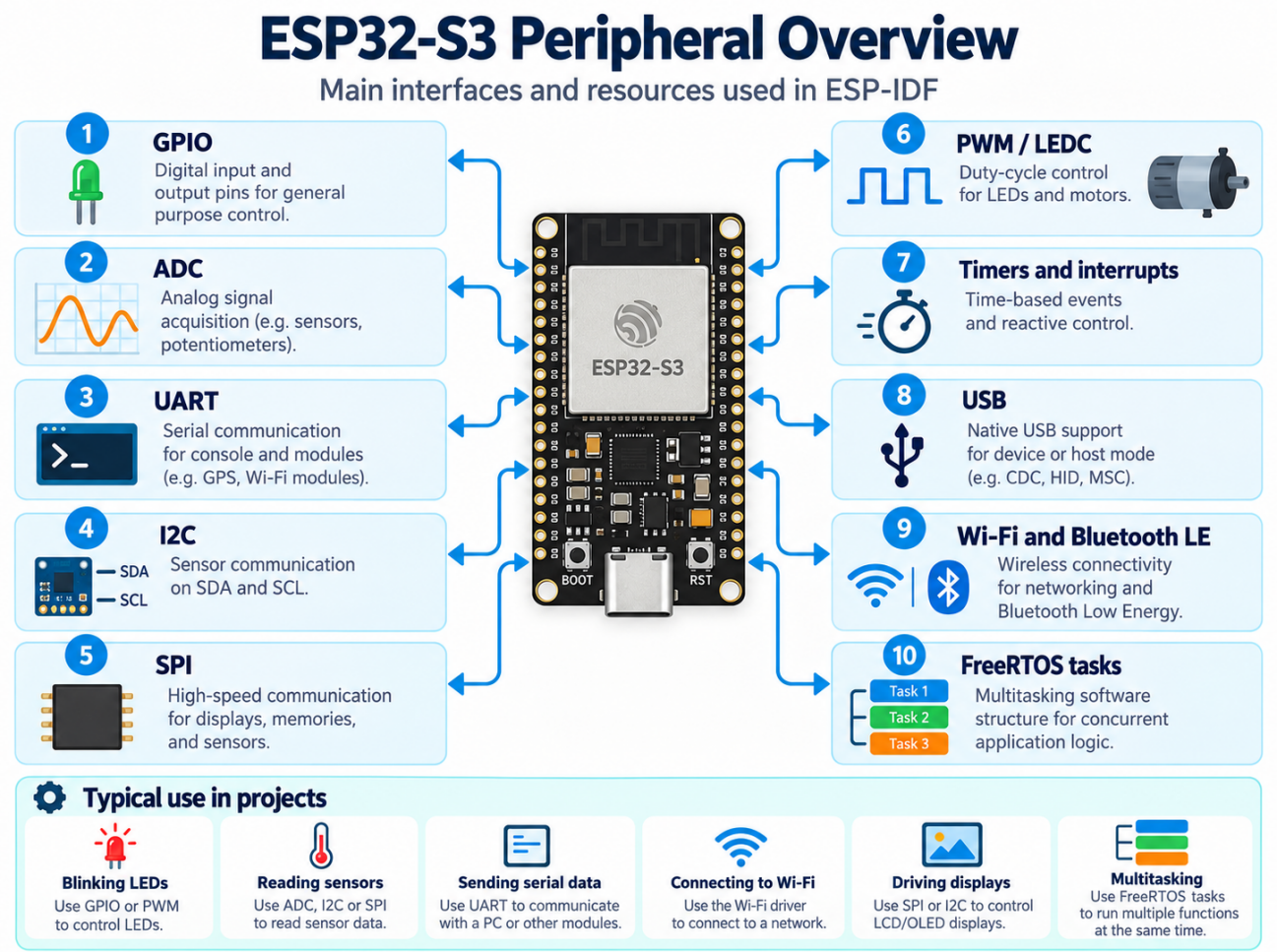


Figure 2. Main ESP32-S3 peripherals exposed through ESP-IDF.

Voltage level: ESP32-S3 GPIOs are 3.3 V logic. Do not assume 5 V tolerance when connecting external modules.

3 Installing ESP-IDF on Windows, macOS, and Linux

A correct ESP-IDF installation must provide the framework source code, target toolchain, Python environment, build tools, flashing utilities, and an activated terminal where idf.py works. The installation procedure differs slightly by operating system, but the final development workflow is the same.

3.1 Common prerequisites

- ESP32-S3 development board
- USB data cable suitable for the board
- Internet access
- Administrator privileges when required
- Visual Studio Code if the graphical workflow will be used

3.2 Windows installation

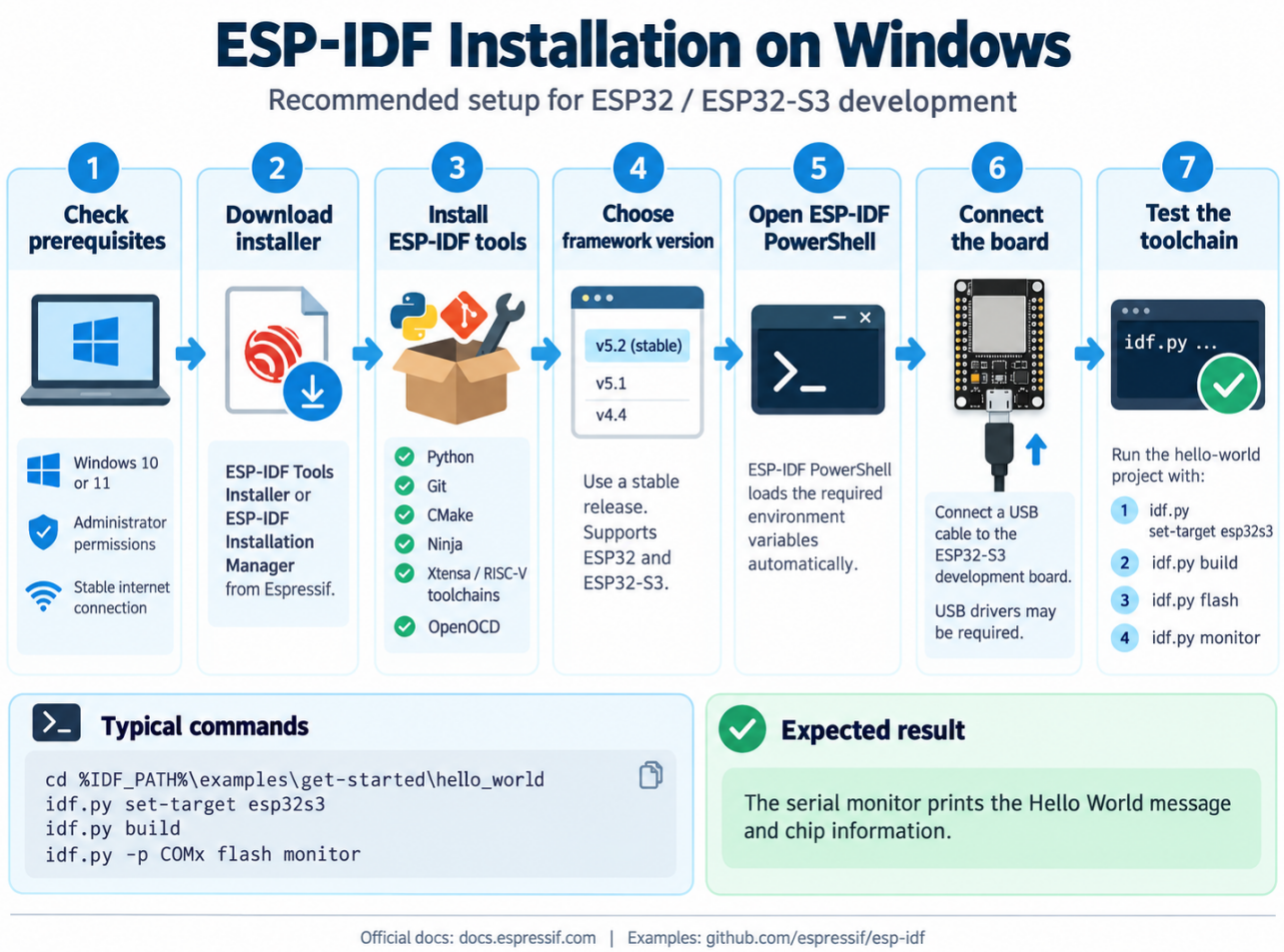


Figure 3. Recommended ESP-IDF installation flow on Windows.

1. Download the current ESP-IDF Installation Manager from Espressif.
2. Run the installer and select the stable ESP-IDF version used by the course.
3. Allow the installer to set up Python, Git, CMake, Ninja, OpenOCD, and the Xtensa/RISC-V toolchains.
4. Open the ESP-IDF PowerShell/terminal installed by Espressif.
5. Connect the ESP32-S3 and identify the COM port in Device Manager.
6. Verify the environment with idf.py --version.

```
idf.py --version
idf.py --help
```

Common Windows issue: If the board powers on but no COM port appears, first check the USB cable and then the USB-to-UART or native-USB driver used by the board.

3.3 macOS installation

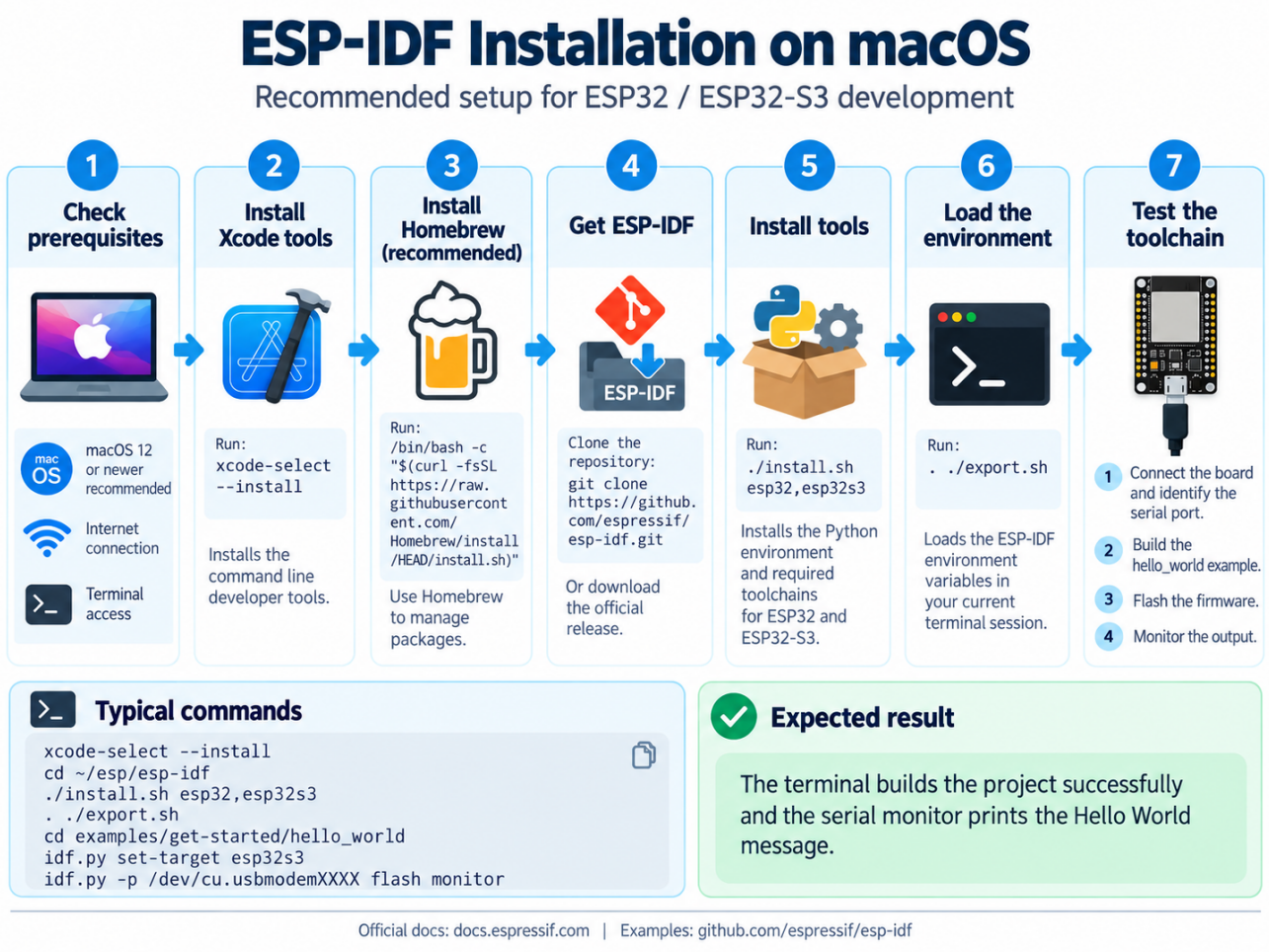


Figure 4. ESP-IDF installation flow on macOS.

A practical manual workflow on macOS uses the Xcode command-line tools, the ESP-IDF repository, the install script, and export.sh. Homebrew is optional but convenient for package management.

```
xcode-select --install
mkdir -p ~/esp
cd ~/esp
git clone --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh esp32,esp32s3
. ./export.sh
idf.py --version
ls /dev/cu.*
```

Shell environment: The command . ./export.sh modifies the current shell session. If the environment is not loaded, idf.py and the toolchain may not be found.

3.4 Linux installation

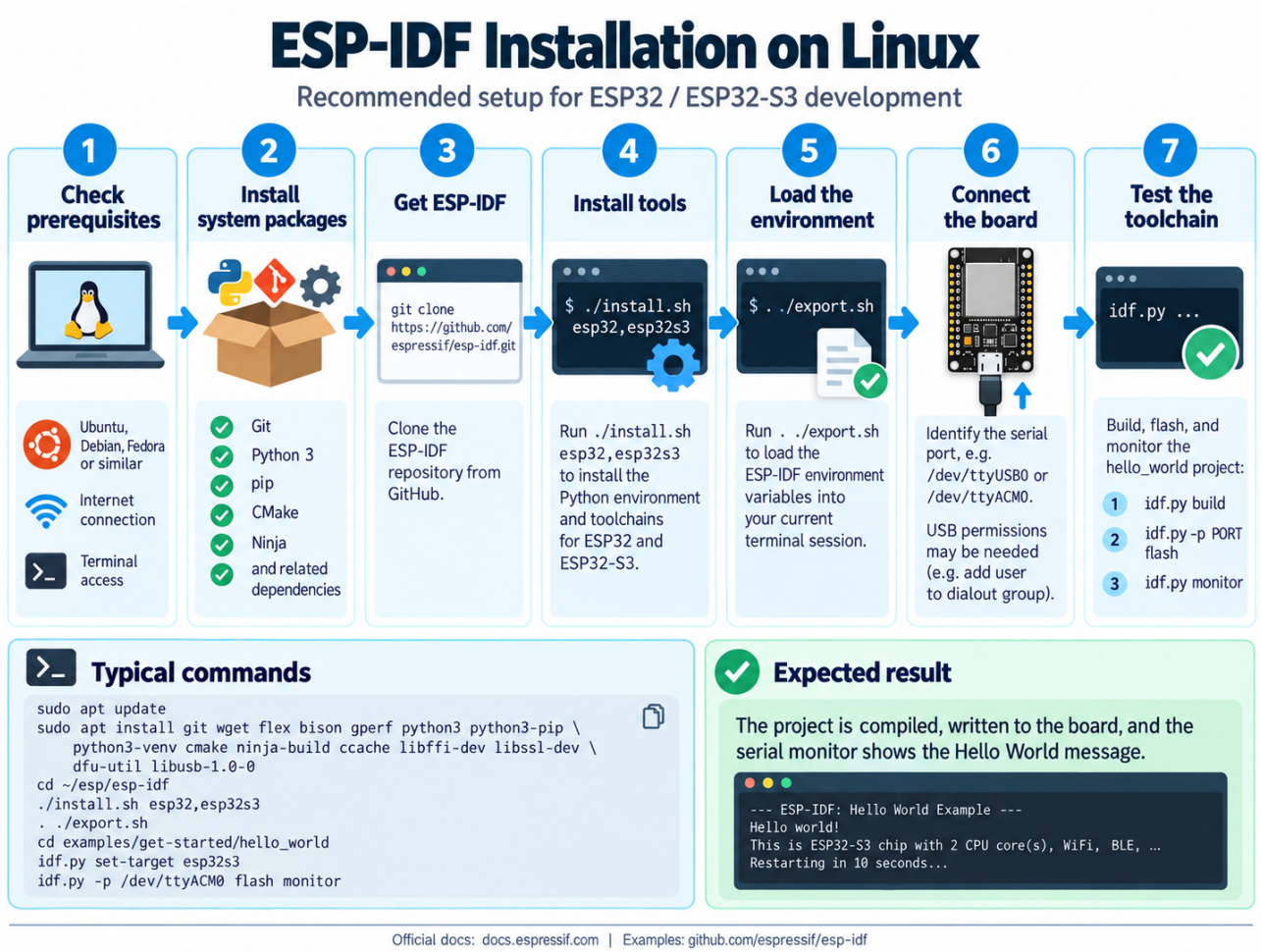


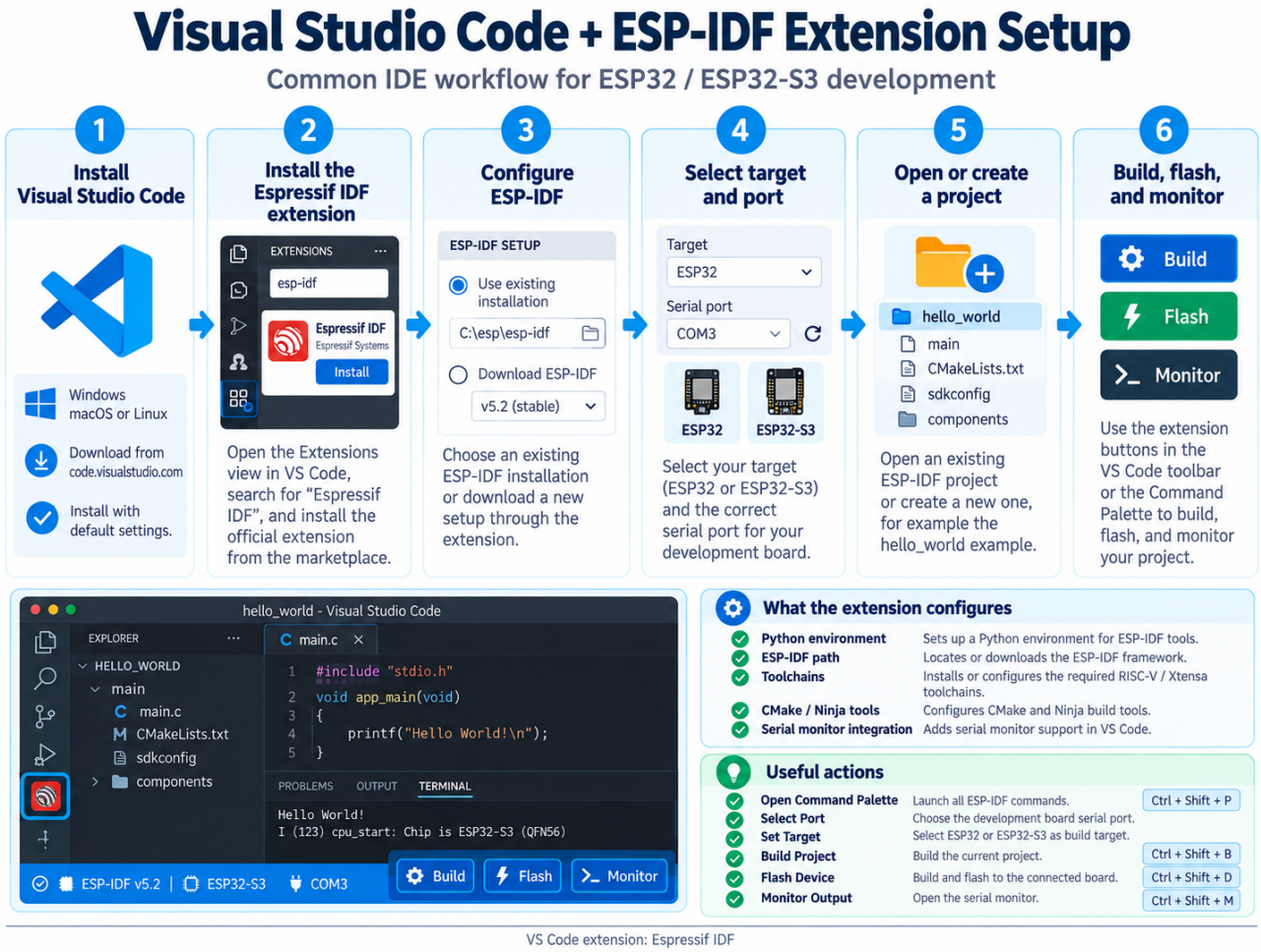
Figure 5. ESP-IDF installation flow on Linux.

```
sudo apt update
sudo apt install git wget flex bison gperf python3 python3-pip python3-venv \
cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0
mkdir -p ~/esp
cd ~/esp
git clone --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh esp32,esp32s3
. ./export.sh
idf.py --version
ls /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
```

Serial permissions: On Ubuntu/Debian, a common solution for serial-port permission errors is sudo usermod -aG dialout $USER followed by logging out and back in.

4 Visual Studio Code and the ESP-IDF Extension

The official Espressif IDF extension provides an integrated workflow for configuring the framework, choosing a target, selecting a serial port, building, flashing, monitoring, and debugging. Students should still learn the equivalent idf.py commands because they make the development process reproducible and easier to troubleshoot.



VS Code extension: Espressif IDF

Figure 6. Visual Studio Code with the official Espressif IDF extension.

7. Install Visual Studio Code.
8. Install the official "Espressif IDF" extension from the Extensions view.
9. Select an existing ESP-IDF installation or let the extension configure one.
10. Select target esp32s3.
11. Select the correct serial port.
12. Use Build, Flash, Monitor, menuconfig, and debugging commands from the extension.

4.1 Ubuntu: Recommended Classroom Setup for Reliable VS Code Detection

For current ESP-IDF releases, especially ESP-IDF v6.x, the most reliable classroom workflow on Ubuntu is to use the **ESP-IDF Installation Manager (EIM)** and let the VS Code extension discover the installation through EIM metadata. Espressif uses `eim_idf.json` as the discovery database for installations created by EIM.

Recommended VS Code package on Ubuntu

For laboratory computers, this guide recommends the official Microsoft `.deb` package of Visual Studio Code rather than the Snap package. Snap is an official VS Code distribution method, and recent versions of the Espressif extension include Snap-related fixes, but recent Ubuntu reports have still shown EIM GUI launch and setup-selection problems under Snap. The native `.deb` package avoids an additional sandbox layer and is therefore the preferred course configuration. This is a course reliability recommendation, not a statement that Snap can never work.

4.1.1 Step A: Check how VS Code is installed

Run:

```
command -v code
readlink -f "$(command -v code)"
snap list code 2>/dev/null || true
```

If `code` resolves to `/snap/bin/code`, the Snap package is being used. For the controlled laboratory setup, install the official Debian package from Microsoft instead. The standard Ubuntu installation form is:

```
sudo apt install ./code_*.deb
```

The official Microsoft package can also configure its APT repository so future updates arrive through `apt`.

4.1.2 Step B: Install EIM from Espressif's APT repository

On Ubuntu, installing EIM through Espressif's APT repository is a clean way to make the installer available system-wide while keeping each student's ESP-IDF installation in the user's home directory.

```
sudo apt update
sudo apt install -y curl

sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://dl.espressif.com/dl/eim/eim.gpg \
  -o /etc/apt/keyrings/eim.gpg
```

```
sudo chmod 0644 /etc/apt/keyrings/eim.gpg

sudo curl -fsSL https://dl.espressif.com/dl/eim/eim.sources \
  -o /etc/apt/sources.list.d/espressif.sources

sudo apt update
sudo apt install -y eim
```

Do **not** run the ESP-IDF installation itself with `sudo`. Run EIM as the normal student account:

```
eim --version
eim wizard
```

The wizard checks prerequisites, asks for the ESP-IDF version and target features, and creates the required toolchain and Python environment.

4.1.3 Step C: Verify the installation before opening VS Code

After EIM finishes, verify that the metadata file exists:

```
test -f "$HOME/.espressif/tools/eim_idf.json" \
  && echo "EIM metadata: OK" \
  || echo "EIM metadata: MISSING"

ls -l "$HOME/.espressif/tools/eim_idf.json"
find "$HOME/.espressif" -maxdepth 4 -type d -name esp-idf -print
```

Then activate the installed ESP-IDF environment using the activation script created by EIM, or open an EIM-activated terminal, and confirm:

```
idf.py --version
python --version
```

If `idf.py -version` does not work here, fix the ESP-IDF installation before attempting to configure the VS Code extension.

4.1.4 Step D: Configure the extension in VS Code

Install the official **Espressif IDF** extension, then open **Preferences: Open User Settings (JSON)**. On Ubuntu, the safest explicit metadata setting is:

```
"idf.eimIdfJsonPath": "${env:HOME}/.espressif/tools/eim_idf.json"
```

Do not use a raw `~` or `$HOME` in extension path settings. The extension documentation states that those forms are not expanded; use an absolute path or the VS Code form `${env:HOME}`.

Next:

1. Press `Ctrl+Shift+P`.
2. Run **ESP-IDF: Select Current ESP-IDF Version**.
3. Select the EIM-installed ESP-IDF version.
4. Run **Developer: Reload Window**.
5. Run **ESP-IDF: Doctor Command**.

6. Run **ESP-IDF: Open ESP-IDF Terminal** and verify `idf.py -version` in that integrated terminal.

At this point the extension should be using the same ESP-IDF setup that EIM registered for the user.

4.1.5 Step E: Ubuntu workaround if the installation is visible to EIM but not selected by VS Code

A recent issue in the official Espressif extension repository documented an Ubuntu 24.04 case where EIM had installed ESP-IDF correctly and `eim_idf.json` was valid, but the extension did not persist the selected setup correctly. The working workaround was to set `idf.currentSetup` to the **actual ESP-IDF directory path**, not to the installation identifier.

First inspect the EIM metadata file and identify the installed ESP-IDF path:

```
cat "$HOME/.espressif/tools/eim_idf.json"
```

Then, in VS Code settings, use the real path shown by EIM, for example:

```
"idf.eimIdfJsonPath": "${env:HOME}/.espressif/tools/eim_idf.json",
"idf.currentSetup": "/home/student/.espressif/v6.x.x/esp-idf"
```

Replace the example with the actual path on the computer. After saving the settings, run **Developer: Reload Window** and then **ESP-IDF: Doctor Command** again.

Why this Ubuntu procedure is more robust

It removes several common sources of ambiguity at once: ESP-IDF is installed by EIM, the extension reads the default EIM database, VS Code is not isolated by a Snap sandbox, the selected setup is checked with Doctor, and the integrated ESP-IDF terminal is used as the final verification point.

4.1.6 Step F: Serial and debug permissions on Ubuntu

For normal USB serial flashing, add the student account to `dialout` if required:

```
sudo usermod -aG dialout "$USER"
```

Log out and back in after changing group membership. For JTAG/OpenOCD debugging, Linux may also require OpenOCD udev rules. This is a debugging requirement and is separate from basic `build`, `flash`, and `monitor` use.

4.1.7 Classroom acceptance test

Before considering the Ubuntu workstation ready for class, verify all five items:

1. `idf.py -version` works in a normal EIM-activated terminal.
2. `ESP-IDF: Select Current ESP-IDF Version` shows the installed setup.
3. `ESP-IDF: Doctor Command` reports valid IDF, tools, and Python paths.
4. `ESP-IDF: Open ESP-IDF Terminal` also runs `idf.py -version`.
5. A `hello_world` project can be built, flashed, and monitored from VS Code.

Only after these checks pass should the machine be treated as correctly configured for the course.

4.2 Troubleshooting: VS Code Cannot Detect an Existing ESP-IDF Installation

A common installation problem is that ESP-IDF works from a terminal, but the Espressif IDF extension in Visual Studio Code reports that no ESP-IDF setup is available. This usually means that the framework itself is installed correctly, but the extension cannot locate the ESP-IDF setup metadata or the Python/tools environment associated with that installation.

Recommended diagnostic order

Do not reinstall everything immediately. First verify the command-line installation, then ask the extension to select the installed setup, then check the EIM metadata file, and only use manual path configuration as a fallback.

4.2.1 Step 1: Verify ESP-IDF outside Visual Studio Code

Open an ESP-IDF-enabled terminal and run:

```
idf.py --version
python --version
```

If `idf.py -version` fails, the problem is not Visual Studio Code yet: the ESP-IDF environment has not been activated correctly. On Linux/macOS, for a manual installation, activate it with:

```
cd ~/esp/esp-idf
. ./export.sh
idf.py --version
```

For an EIM installation, use the activation script created by the Installation Manager. The exact script name depends on the installed ESP-IDF version.

4.2.2 Step 2: Ask the extension to select the installed version

In Visual Studio Code:

1. Press `F1` or `Ctrl+Shift+P` to open the Command Palette.
2. Run **ESP-IDF: Select Current ESP-IDF Version**.
3. Select the ESP-IDF installation that should be used by the current workspace.
4. Run **ESP-IDF: Doctor Command** and inspect the report.

The selected installation is stored by the extension as the current ESP-IDF setup for the workspace. The Doctor command is especially useful because it reports missing paths, Python environments, tools, and configuration problems.

4.2.3 Step 3: If the list is empty, check the EIM metadata file

Recent versions of the extension automatically discover ESP-IDF installations created with the ESP-IDF Installation Manager (EIM) by reading `eim_idf.json`. The default locations are:

Operating system	Default EIM metadata path
Windows	`C:\Espressif\tools\eim_idf.json`
Linux	`$HOME/.espressif/tools/eim_idf.json`
macOS	`$HOME/.espressif/tools/eim_idf.json`

Check that the file exists. If it is stored elsewhere, open Visual Studio Code Settings and set:

```
"idf.eimIdfJsonPath": "/absolute/path/to/eim_idf.json"
```

Then restart Visual Studio Code and run **ESP-IDF: Select Current ESP-IDF Version** again.

4.2.4 Step 4: Manual configuration when EIM detection still fails

If ESP-IDF was installed manually, or if EIM discovery is not working, the extension can be pointed to the installation explicitly. First obtain the required environment variables from a working ESP-IDF terminal:

```
# Linux/macOS manual installation
source /home/user/esp-idf/export.sh
python /home/user/esp-idf/tools/idf_tools.py export --format key-value
```

Typical values are:

- `IDF_PATH`: location of the ESP-IDF source tree.
- `IDF_TOOLS_PATH`: directory containing Espressif tools.
- `IDF_PYTHON_ENV_PATH`: Python virtual environment used by ESP-IDF.

Open **Preferences: Open Settings (JSON)** and add, using the real paths from your computer:

```
"idf.customExtraVars": {
    "IDF_PATH": "/home/user/esp-idf",
    "IDF_TOOLS_PATH": "/home/user/.espressif",
    "IDF_PYTHON_ENV_PATH":
        "/home/user/.espressif/python_env/idf6.x_py3.x_env"
}
```

On Windows, use absolute Windows paths, for example:

```
"idf.customExtraVars": {
    "IDF_PATH": "C:\\Espressif\\frameworks\\esp-idf-v6.1",
    "IDF_TOOLS_PATH": "C:\\Espressif"
}
```

Do not mix two configuration methods

If custom ESP-IDF environment variables are configured manually in the VS Code settings, they can override a setup selected with **ESP-IDF: Select Current ESP-IDF Version**. If switching back to EIM-based detection, remove the manual `IDF_PATH`, `IDF_TOOLS_PATH`, and `IDF_PYTHON_ENV_PATH` values first.

4.2.5 Step 5: Check User Settings versus Workspace Settings

VS Code settings can exist at User, Workspace, or Workspace Folder scope. A stale project-specific `.vscode/settings.json` can therefore override a correct global configuration. If detection behaves differently between projects, inspect:

- `<project>/.vscode/settings.json`
- the current `.code-workspace` file, if one is used
- the global VS Code `settings.json`

Also avoid using raw `~`, `$HOME`, or `%VARNAME%` inside ESP-IDF extension path settings. Use an absolute path, or the VS Code form `${env:HOME}` when supported.

4.2.6 Step 6: WSL, SSH, containers, and remote VS Code

When VS Code is connected to WSL, SSH, a Dev Container, or another remote environment, the ESP-IDF extension runs in that remote environment. Therefore, ESP-IDF must be installed or visible there, not only on the host operating system. Install/enable the extension in the remote side and verify `idf.py -version` from the integrated terminal of that same remote session.

4.2.7 Quick recovery checklist

1. Confirm `idf.py -version` works in an ESP-IDF terminal.
2. Run **ESP-IDF: Select Current ESP-IDF Version**.
3. Run **ESP-IDF: Doctor Command**.
4. Confirm that `eim_idf.json` exists in the expected location.
5. Set `idf.eimIdfJsonPath` if the metadata file is stored elsewhere.
6. Check for stale values in User/Workspace `settings.json`.
7. Use `idf.customExtraVars` only as a manual fallback.
8. Restart VS Code after changing setup paths.

Official references

The current ESP-IDF VS Code extension documentation describes EIM discovery, `idf.eimIdfJsonPath`, `ESP-IDF: Select Current ESP-IDF Version`, and the Doctor command. See https://github.com/espressif/vscode-esp-idf-extension/blob/master/docs_espressif/en/installation.rst and https://github.com/espressif/vscode-esp-idf-extension/blob/master/docs_espressif/en/settings.rst.

5 ESP-IDF Project Structure and Development Workflow

ESP-IDF projects are component-based and use CMake. A minimal project contains a top-level CMakeLists.txt file, a main component, source code, and a generated sdkconfig file.

```
my_project/
|—- CMakeLists.txt
|—- sdkconfig
|—- sdkconfig.defaults # optional
|—- partitions.csv # optional
|—- main/
| |—- CMakeLists.txt
| |—- main.c
|—- components/ # optional reusable components
```

```
# Top-level CMakeLists.txt
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(my_project)

# main/CMakeLists.txt
idf_component_register(SRCS "main.c" INCLUDE_DIRS ".")
```

ESP-IDF starts the user application through app_main(). The framework already runs on FreeRTOS, so app_main() executes in a task context and can create additional tasks, queues, timers, event groups, or synchronization objects.

```
#include "esp_log.h"

static const char *TAG = "APP";

void app_main(void)
{
ESP_LOGI(TAG, "Application started");
}
```

Command	Purpose
idf.py set-target esp32s3	Select the ESP32-S3 build target
idf.py menuconfig	Open the project configuration menu
idf.py build	Configure and compile the application
idf.py -p PORT flash	Write the firmware to the board
idf.py -p PORT monitor	Open the serial monitor
idf.py -p PORT flash monitor	Flash and immediately open the monitor
idf.py fullclean	Remove generated build files
idf.py size	Inspect binary memory usage

6 GPIO, Pin Assignment, Remapping, and Conflicts

The ESP32-S3 has a very flexible pin-routing architecture. Through the GPIO matrix and IO MUX, many internal peripheral signals can be routed to different physical GPIOs. This means that protocols such as I2C, UART, and

general-purpose SPI are usually not limited to one fixed pin set. However, some pins have special roles and should be selected carefully.

6.1 What is a GPIO?

A **GPIO (General-Purpose Input/Output)** is a programmable digital pin that software can configure to read or drive logic levels. A GPIO can be used as a simple digital input or output, or it can be connected internally to a peripheral such as UART, I2C, SPI, LEDC/PWM, RMT, or a timer capture function through the ESP32-S3 GPIO matrix.

A useful mental model is that the **physical pin** is only the electrical connection to the outside world, while the **GPIO matrix** is the internal routing network that connects that pin to a selected peripheral signal. This is why many ESP32-S3 peripheral signals can be moved to different GPIO numbers without changing the communication protocol itself.

Table 14: Common GPIO configurations in ESP-IDF.

Mode	Meaning	Typical use
Input	Reads a digital logic level from the external circuit.	Buttons, digital sensors, interrupt inputs.
Output	Drives the pin HIGH or LOW.	LEDs, enables, chip-select lines.
Input/output	Allows reading and driving the same pin when the protocol requires it.	Bidirectional buses and open-drain interfaces.
Pull-up	Weak internal resistor biases the pin toward logic HIGH.	Buttons, open-drain signals in simple experiments.
Pull-down	Weak internal resistor biases the pin toward logic LOW.	Inputs that must not float when disconnected.
Interrupt input	Generates an event on rising, falling, or level conditions.	Buttons, alarms, encoders, pulse signals.
Peripheral signal	Routes an internal hardware peripheral to the selected GPIO.	UART TX/RX, I2C SDA/SCL, SPI signals, PWM.

Floating inputs

An unconnected digital input can randomly read HIGH or LOW because it is electrically floating. Use an internal or external pull-up/pull-down resistor when the external circuit does not actively define the input level.

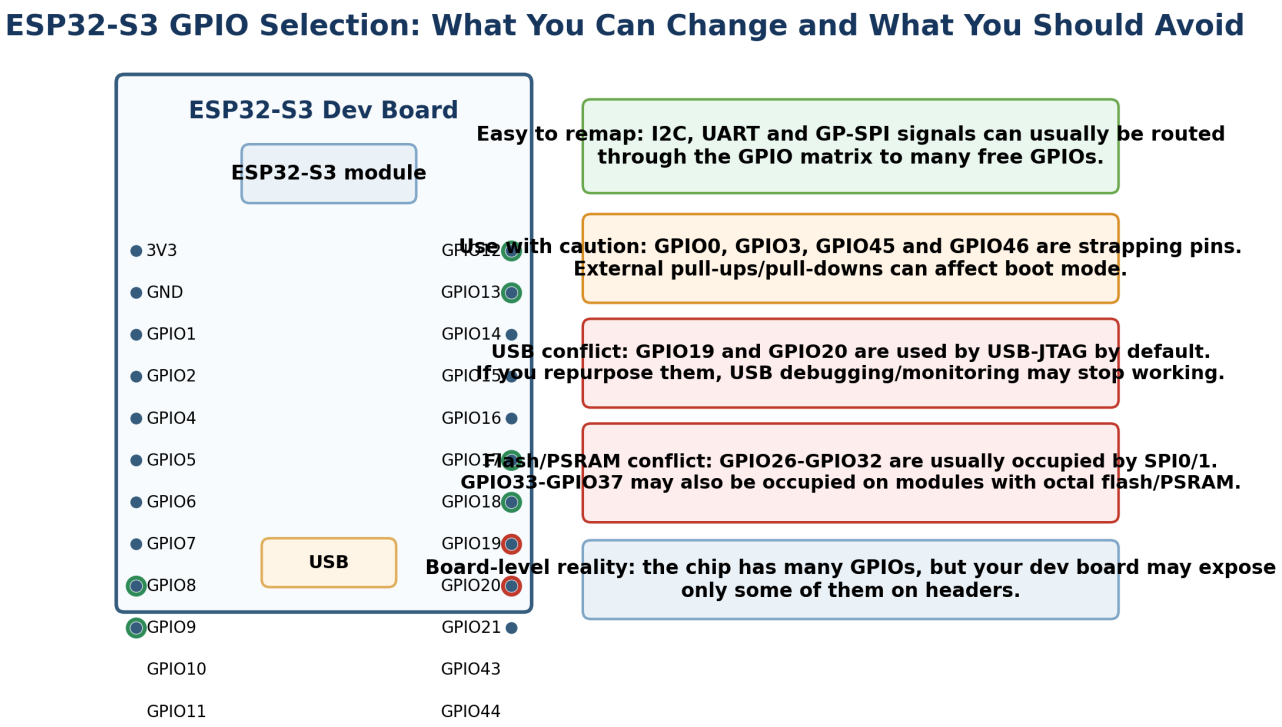


Figure 7. Practical GPIO selection rules for ESP32-S3.

Pin group	Meaning / restriction
GPIO0, GPIO3, GPIO45, GPIO46	Strapping pins; external circuitry can affect boot behavior
GPIO19, GPIO20	Used by USB-JTAG by default; remapping can disable USB debugging/serial
GPIO26-GPIO32	Usually associated with SPI0/1 flash/PSRAM and not recommended as general-purpose GPIO
GPIO33-GPIO37	May also be occupied when octal flash/PSRAM is used
GPIO1-GPIO20	Many are ADC-capable, but exact ADC channel mapping must be checked
Other exposed GPIOs	Often available for peripheral remapping, depending on the development board

6.2 Example classroom pin plan

Function	Example pins	Reason
I2C	SDA=GPIO8, SCL=GPIO9	Simple and easy to document
Alternative I2C	SDA=GPIO4, SCL=GPIO5	Useful for teaching remapping
Application UART	TX=GPIO17, RX=GPIO18	Keeps an external module separate from the console
GP-SPI	CS=10, MOSI=11, SCK=12, MISO=13	Compact group for microSD/display labs

Function	Example pins	Reason
ADC	GPIO1-GPIO18 as available	Many are analog-capable; verify channel mapping

6.3 Changing I2C pins

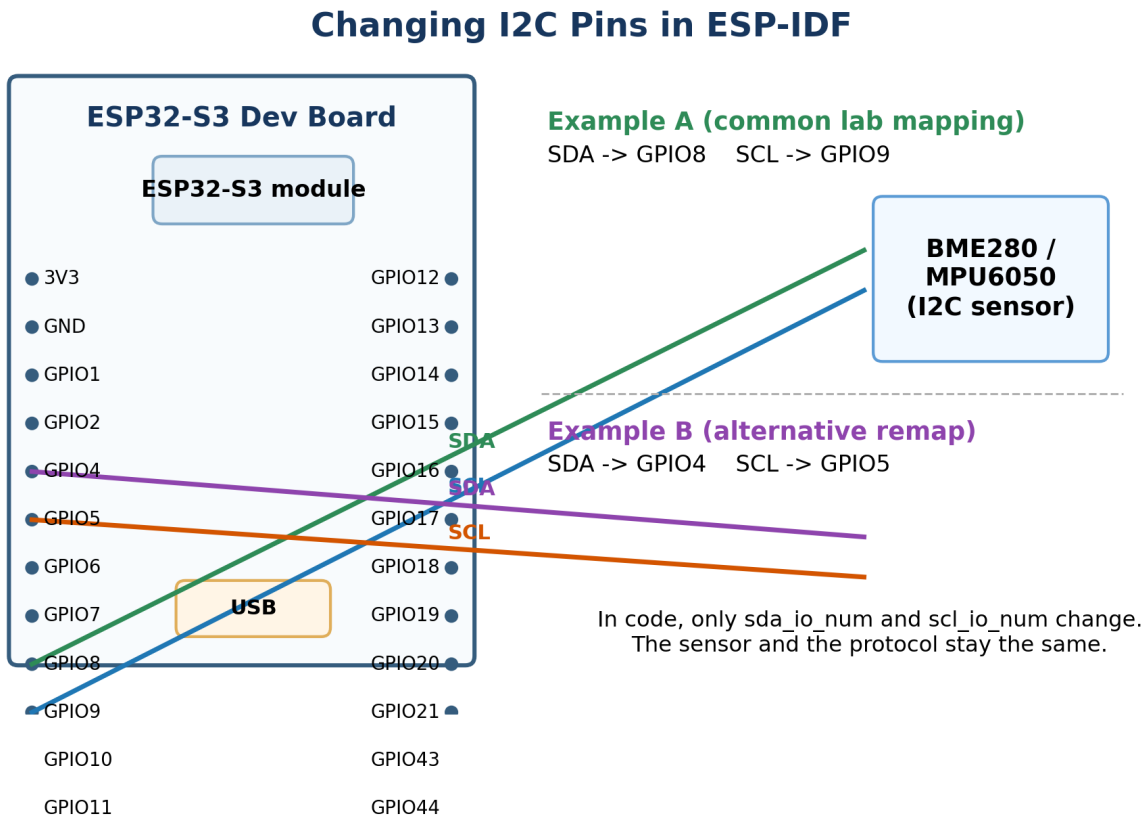


Figure 8. The same I2C sensor can be moved to another GPIO pair by changing wiring and configuration.

```
// Option A: I2C on GPIO8/GPIO9
i2c_master_bus_config_t bus_cfg = {
.i2c_port = I2C_NUM_0,
.sda_io_num = GPIO_NUM_8,
.scl_io_num = GPIO_NUM_9,
.clk_source = I2C_CLK_SRC_DEFAULT,
.glitch_ignore_cnt = 7,
.flags.enable_internal_pullup = true,
};

// Option B: remap the same bus to GPIO4/GPIO5
bus_cfg.sda_io_num = GPIO_NUM_4;
bus_cfg.scl_io_num = GPIO_NUM_5;
```

Important: Changing the software pin numbers is not enough. The physical wires must also be moved to the same GPIOs.

6.4 Typical conflicts to recognize

Conflict	Symptom	Recommendation
Boot/strapping	Board boots incorrectly or enters download mode unexpectedly	Avoid strapping pins in beginner labs
USB-JTAG	USB serial/debugging disappears	Keep GPIO19/GPIO20 free if USB-JTAG is required
Flash/PSRAM	Instability or failure to boot	Treat flash/PSRAM-connected GPIOs as reserved
I2C address conflict	Two devices respond at the same address	Change address pin/configuration or use another bus
SPI chip-select conflict	Two SPI devices respond simultaneously	Assign a unique CS pin to every device
UART console conflict	External device corrupts or receives console traffic	Use a separate application UART

7 Serial Communication Interfaces: UART, I2C, and SPI

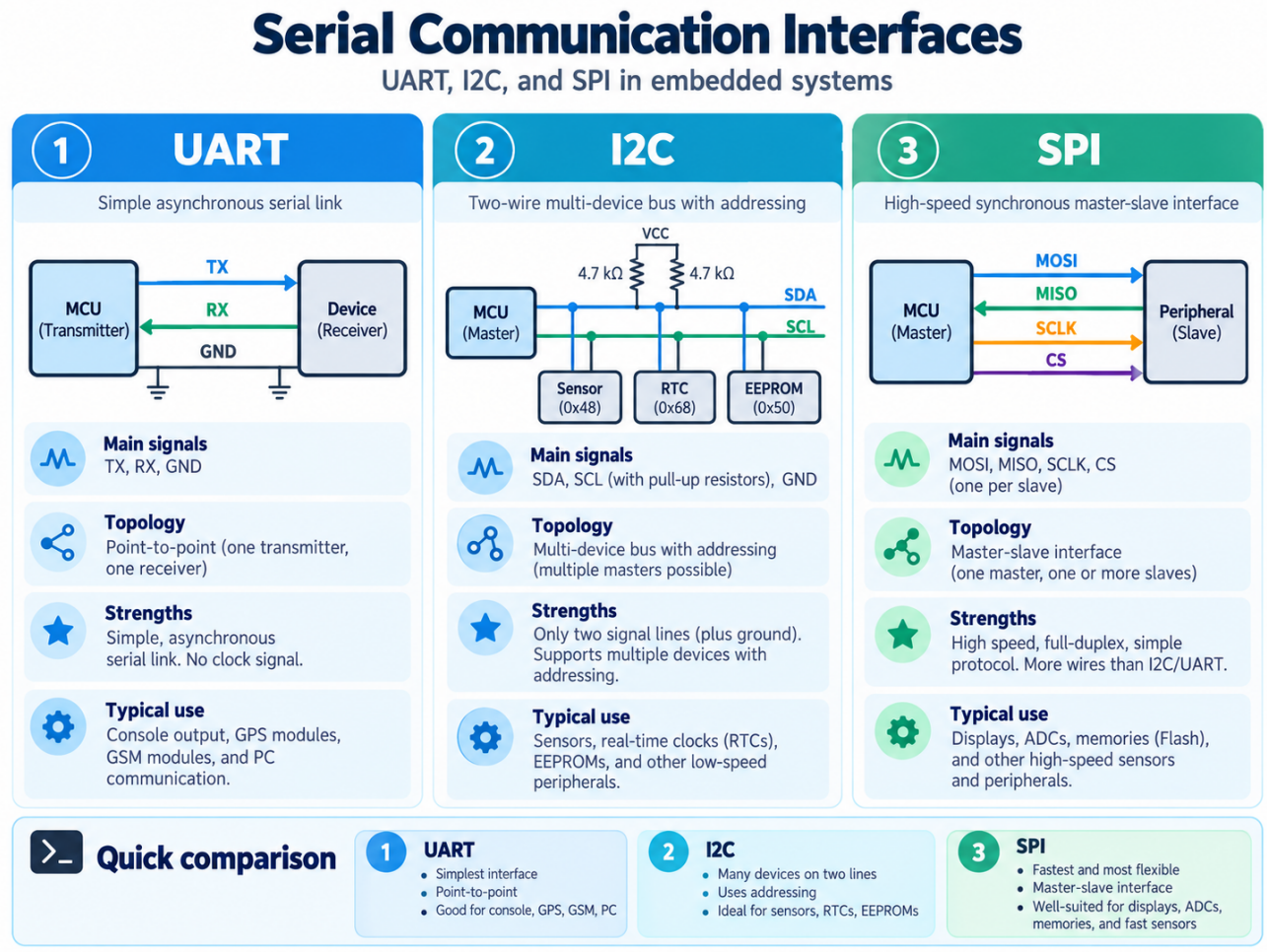


Figure 9. Conceptual comparison of UART, I2C, and SPI.

Interface	Clocking	Typical signals	Topology	Typical use
UART	Asynchronous	TX, RX, GND	Point-to-point	Console, GPS, modem, MCU-to-MCU
I2C	Synchronous	SDA, SCL, GND	Addressed multi-device bus	Sensors, RTC, EEPROM, IMU
SPI	Synchronous	MOSI, MISO, SCK, CS, GND	Controller with one or more devices	Displays, flash, microSD, fast sensors

7.1 UART

UART is an asynchronous serial interface. Both endpoints must agree on baud rate, data bits, parity, and stop bits. There is no shared clock. UART is frequently used for console output, GNSS/GPS receivers, cellular modules, and communication between microcontrollers.

```
#include "driver/uart.h"
#define UART_PORT UART_NUM_1
#define UART_TX GPIO_NUM_17
#define UART_RX GPIO_NUM_18

void init_uart(void)
{
uart_config_t cfg = {
.baud_rate = 115200,
.data_bits = UART_DATA_8_BITS,
.parity = UART_PARITY_DISABLE,
.stop_bits = UART_STOP_BITS_1,
.flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
.source_clk = UART_SCLK_DEFAULT,
};
ESP_ERROR_CHECK(uart_driver_install(UART_PORT, 2048, 0, 0, NULL, 0));
ESP_ERROR_CHECK(uart_param_config(UART_PORT, &cfg));
ESP_ERROR_CHECK(uart_set_pin(UART_PORT, UART_TX, UART_RX,
UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE));
}
```

7.2 I2C

I2C is a synchronous, half-duplex, addressed bus using two bidirectional open-drain signals: SDA for data and SCL for clock. Pull-up resistors are required. ESP32-S3 provides two I2C controllers and supports standard and fast mode up to 400 kHz.

```
#include "driver/i2c_master.h"

i2c_master_bus_config_t bus_cfg = {
.i2c_port = I2C_NUM_0,
.sda_io_num = GPIO_NUM_8,
.scl_io_num = GPIO_NUM_9,
.clk_source = I2C_CLK_SRC_DEFAULT,
.glitch_ignore_cnt = 7,
.flags.enable_internal_pullup = true,
};

i2c_master_bus_handle_t bus;
ESP_ERROR_CHECK(i2c_new_master_bus(&bus_cfg, &bus));
```

Pull-ups: Internal pull-ups are useful for simple tests, but external pull-ups are normally preferred for robust I2C buses. Common values are around 2 kΩ to 10 kΩ depending on bus capacitance and speed.

7.3 SPI

SPI is a synchronous, high-speed interface. A typical bus uses SCK, MOSI, MISO, and one chip-select line per device. Multiple peripherals can share SCK/MOSI/MISO as long as each device has its own CS signal and the devices are electrically compatible.

```
#include "driver/spi_master.h"

spi_bus_config_t buscfg = {
.mosi_io_num = GPIO_NUM_11,
.miso_io_num = GPIO_NUM_13,
.sclk_io_num = GPIO_NUM_12,
.quadwp_io_num = -1,
.quadhd_io_num = -1,
};
ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &buscfg, SPI_DMA_CH_AUTO));
```

8 ADC Fundamentals and Example

An ADC converts an analog voltage into a digital value. The ESP32-S3 includes two 12-bit SAR ADCs. In embedded sensor systems, students should understand resolution, sampling, attenuation/input range, quantization, noise, and calibration.

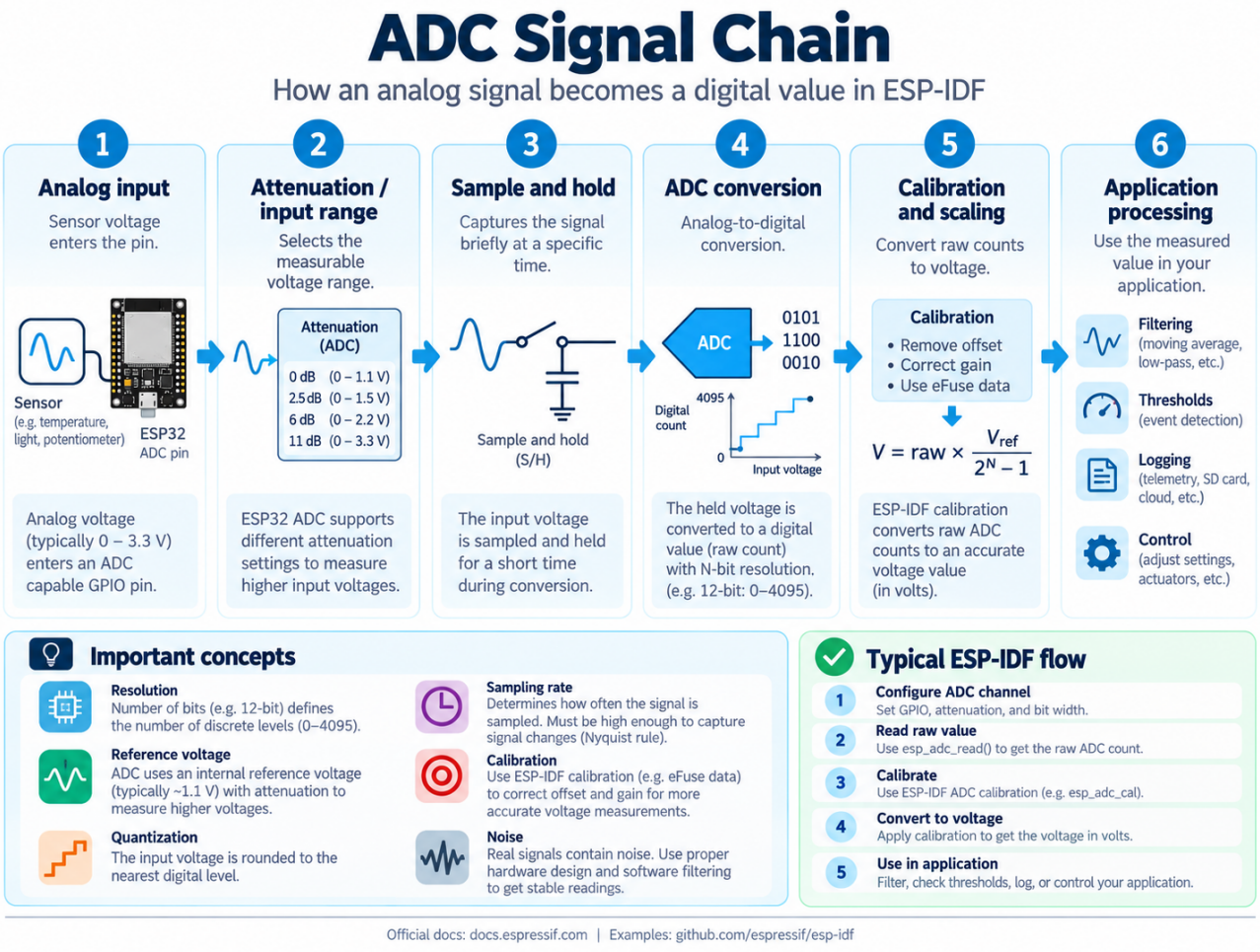


Figure 10. ADC signal chain from sensor voltage to application data.

Concept	Definition
Resolution	Number of digital levels. A 12-bit result has 4096 nominal codes (0-4095).
Attenuation	ADC front-end setting that changes the usable input-voltage range.
Sampling rate	Number of samples acquired per second.
Quantization	Mapping of a continuous analog value to a discrete digital code.
Calibration	Correction used to estimate voltage more accurately from raw ADC readings.
Aliasing	Distortion caused by sampling a signal too slowly for its bandwidth.

```
#include "esp_adc/adc_oneshot.h"

adc_oneshot_unit_handle_t adc_handle;
adc_oneshot_unit_init_cfg_t init_cfg = {
.unit_id = ADC_UNIT_1,
};
ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_cfg, &adc_handle));

adc_oneshot_chan_cfg_t chan_cfg = {
.atten = ADC_ATTEN_DB_12,
.bitwidth = ADC_BITWIDTH_DEFAULT,
};

const adc_channel_t channel = ADC_CHANNEL_3;
ESP_ERROR_CHECK(adc_oneshot_config_channel(adc_handle, channel, &chan_cfg));

int raw = 0;
ESP_ERROR_CHECK(adc_oneshot_read(adc_handle, channel, &raw));
```

GPIO vs ADC channel: ADC channel numbers are not the same as GPIO numbers. Always check the ESP32-S3 ADC/GPIO mapping for the selected pin.

9 Interrupts, Watchdogs, Dual-Core Programming, and Blocking Functions

The ESP32-S3 is more than a microcontroller that executes one statement after another. ESP-IDF runs on FreeRTOS and therefore supports concurrent tasks, interrupts, timers, synchronization primitives, watchdogs, and dual-core scheduling. These concepts are essential when firmware must read sensors, communicate, log data, and respond to events without becoming unresponsive.

9.1 Interrupts: reacting to asynchronous events

An **interrupt** is a hardware or software event that temporarily redirects CPU execution to an **Interrupt Service Routine (ISR)**. Instead of continuously polling an input, the CPU can execute other work and respond only when the event occurs. Typical examples are a GPIO edge, a timer expiration, arrival of serial data, or completion of a DMA transfer.

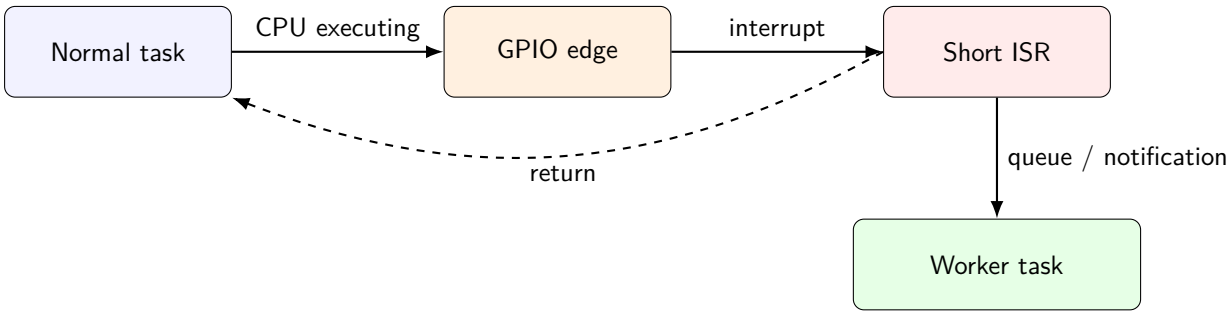


Figure 1: Recommended interrupt pattern: keep the ISR short and defer substantial work to a FreeRTOS task.

ISR design rule

An ISR should do the minimum work necessary: acknowledge the event, capture essential data, and notify a task. Avoid long loops, blocking calls, dynamic allocation, normal queue APIs, and slow formatted printing inside an ISR. Use ISR-safe APIs such as `xQueueSendFromISR()` or task notifications when data must be transferred to normal task context.

```c
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "freertos/queue.h"
#include "freertos/task.h"

#define IRQ_GPIO GPIO_NUM_4
static QueueHandle_t gpio_evt_queue;

static void IRAM_ATTR gpio_isr_handler(void *arg)
{
    uint32_t gpio_num = (uint32_t)arg;
    BaseType_t hp_task_woken = pdFALSE;
    xQueueSendFromISR(gpio_evt_queue, &gpio_num, &hp_task_woken);
    if (hp_task_woken) {
        portYIELD_FROM_ISR();
    }
}

static void gpio_worker(void *arg)
{
    uint32_t gpio_num;
    while (1) {
        if (xQueueReceive(gpio_evt_queue, &gpio_num, portMAX_DELAY)) {
            // Safe place for logging, processing, I2C access, etc.
            printf("Interrupt on GPIO %lu\n", (unsigned long)gpio_num);
        }
    }
}
```

Listing 1: GPIO interrupt with a worker task.

9.2 Watchdog timers

A **watchdog timer** is a safety mechanism that detects software that has stopped making progress. Firmware must periodically allow the system to run normally or explicitly reset ("feed") the watchdog. If the timeout expires, ESP-IDF can report the problem and, depending on configuration, trigger a panic or reset.

For introductory ESP32-S3 work, distinguish the following concepts:

- **Interrupt Watchdog (IWDT):** detects cases where interrupts remain disabled or critical interrupt handling is blocked for too long.
- **Task Watchdog Timer (TWDT):** detects tasks that monopolize a CPU for too long without yielding, or tasks/users that fail to reset their watchdog subscription.
- **RTC watchdog mechanisms:** provide protection during early boot and low-level startup sequences.

```c
#include "esp_task_wdt.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

static void monitored_task(void *arg)
{
    // Assumes the TWDT is enabled/initialized by project configuration.
    ESP_ERROR_CHECK(esp_task_wdt_add(NULL));

    while (1) {
        do_one_bounded_piece_of_work();
        ESP_ERROR_CHECK(esp_task_wdt_reset());

        // Yield so other tasks and the idle task can execute.
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Listing 2: Basic Task Watchdog pattern.

Do not "fix" a watchdog by simply increasing its timeout

A watchdog warning is often evidence that a task contains a busy loop, a long critical section, a peripheral wait without a timeout, or computation that should be split into smaller pieces. First identify why the task is not yielding; only then decide whether the timeout is genuinely too short.

9.3 Programming both ESP32-S3 cores

The ESP32-S3 has two Xtensa LX7 CPU cores. Under ESP-IDF FreeRTOS, applications normally do not write one `main()` function per core. Instead, the application creates **tasks**. The FreeRTOS SMP scheduler can run ready tasks on either core, while **core affinity** can be used when a task must remain on Core 0 or Core 1.

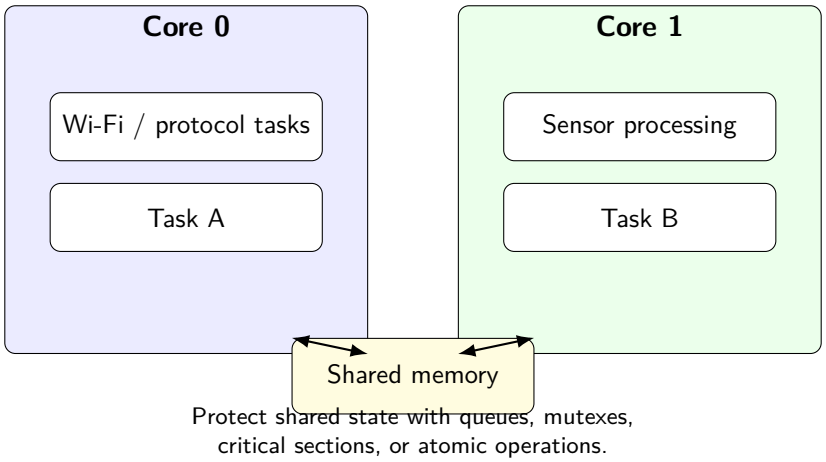


Figure 2: Conceptual dual-core task organization on the ESP32-S3.

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

static void acquisition_task(void *arg)
{
    while (1) {
        read_sensors();
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

static void processing_task(void *arg)
{
    while (1) {
        process_latest_data();
        vTaskDelay(pdMS_TO_TICKS(20));
    }
}

void app_main(void)
{
    xTaskCreatePinnedToCore(acquisition_task, "acq",
                            4096, NULL, 5, NULL, 0);
    xTaskCreatePinnedToCore(processing_task, "proc",
                            4096, NULL, 5, NULL, 1);
}
```

Listing 3: Creating one task on each core.

Pinned vs. unpinned tasks

Use `xTaskCreatePinnedToCore(..., 0)` or `(..., 1)` when there is a real reason to fix a task to one core. Standard `xTaskCreate()` creates a task without a fixed affinity, allowing the scheduler more freedom. Pinning every task is usually unnecessary and can reduce scheduling flexibility.

9.4 Blocking functions and responsive firmware

A **blocking function** does not return immediately; the calling task waits until an event occurs, a delay expires, data arrives, or a timeout is reached. Blocking is not automatically bad. In an RTOS, a well-designed blocking API normally places the current task in the **Blocked** state so that another task can use the CPU.

The important distinction is between **RTOS-aware blocking** and **busy waiting**:

Table 28: Blocking patterns in embedded firmware.

Pattern	Behavior	Recommendation
`vTaskDelay()`	Current task sleeps; CPU runs other ready tasks.	Good for periodic tasks.
Queue/semaphore wait	Task sleeps until data/resource is available or timeout expires.	Preferred synchronization method.
Socket read with timeout	Task waits for network data for a bounded time.	Acceptable in a dedicated communication task.
`portMAX_DELAY`	Task can wait indefinitely for an event.	Good only when another subsystem is guaranteed to eventually signal it.
Busy loop / polling	CPU repeatedly checks a condition without yielding.	Avoid; can waste power and trigger watchdogs.
Long critical section	Scheduler/interrupts may be prevented from progressing.	Keep extremely short.

```
// Bad: consumes CPU continuously and may starve other tasks.
while (!sensor_ready()) {
    // busy wait
}

// Better: poll periodically while allowing other tasks to run.
while (!sensor_ready()) {
    vTaskDelay(pdMS_TO_TICKS(5));
}

// Best when supported: block on an interrupt, queue, semaphore, or event.
xSemaphoreTake(sensor_ready_sem, pdMS_TO_TICKS(1000));
```

Listing 4: Busy waiting versus cooperative waiting.

Design rule for student projects

Every loop that may run for a long time should answer three questions: **Can it yield? Does it have a timeout? What happens if the expected event never occurs?** Thinking this way prevents many watchdog resets and "frozen" firmware problems.

10 Wi-Fi Station and TCP Client

A common IoT pattern is to connect the ESP32-S3 to an access point in station mode and then open an application-layer connection to a server. A TCP client is a good first networking example because it shows the relationship between Wi-Fi connectivity, IP configuration, sockets, server address, and application payloads.

10.1 Wi-Fi station initialization

```
wifi_config_t wifi_config = {
.sta = {
.ssid = "YOUR_SSID",
.password = "YOUR_PASSWORD",
.threshold.authmode = WIFI_AUTH_WPA2_PSK,
},
};
ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
ESP_ERROR_CHECK(esp_wifi_start());
ESP_ERROR_CHECK(esp_wifi_connect());
```

In a complete application, initialize NVS, esp_netif, and the event loop first, then create the default station interface and register Wi-Fi/IP event handlers. Do not open a TCP socket until the system has received an IP address.

10.2 TCP client example

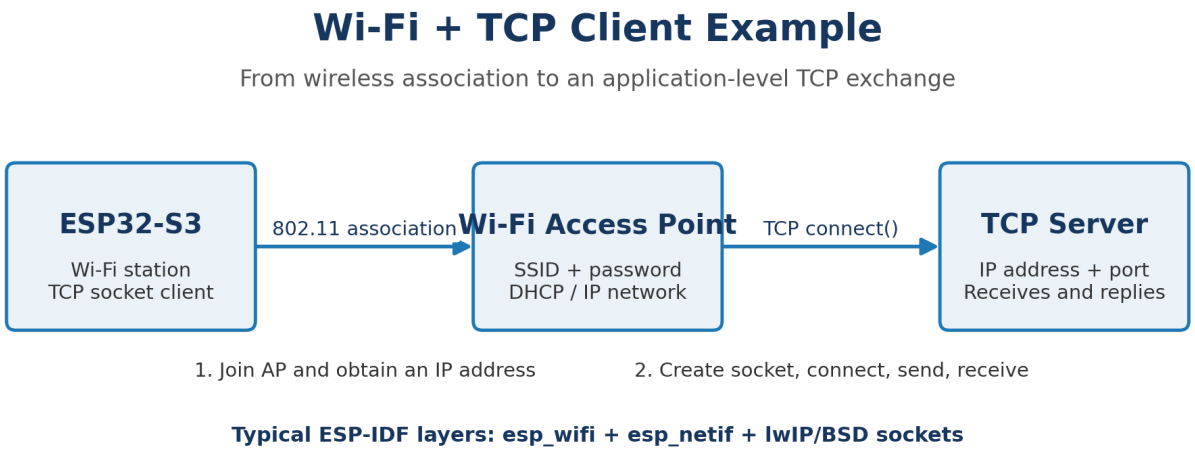


Figure 11. Example data path for an ESP32-S3 TCP client over Wi-Fi.

```
int sock = socket(AF_INET, SOCK_STREAM, IPPROTO_IP);

struct sockaddr_in dest = {
.sin_family = AF_INET,
.sin_port = htons(1234),
};
inet_pton(AF_INET, "192.168.1.100", &dest.sin_addr);

if (connect(sock, (struct sockaddr *)&dest, sizeof(dest)) == 0) {
const char *msg = "Hello from ESP32-S3\n";
send(sock, msg, strlen(msg), 0);
}
close(sock);
```

Classroom test: A laptop on the same network can run a small TCP server or netcat listener. Students can then observe the complete path from sensor node to Wi-Fi router to server.

11 MQTT Publish/Subscribe Example

MQTT is a lightweight publish/subscribe messaging protocol widely used in IoT. Instead of opening a direct application connection to every receiver, devices exchange messages through a broker. A publisher sends data to a topic; subscribers receive messages from topics they follow.

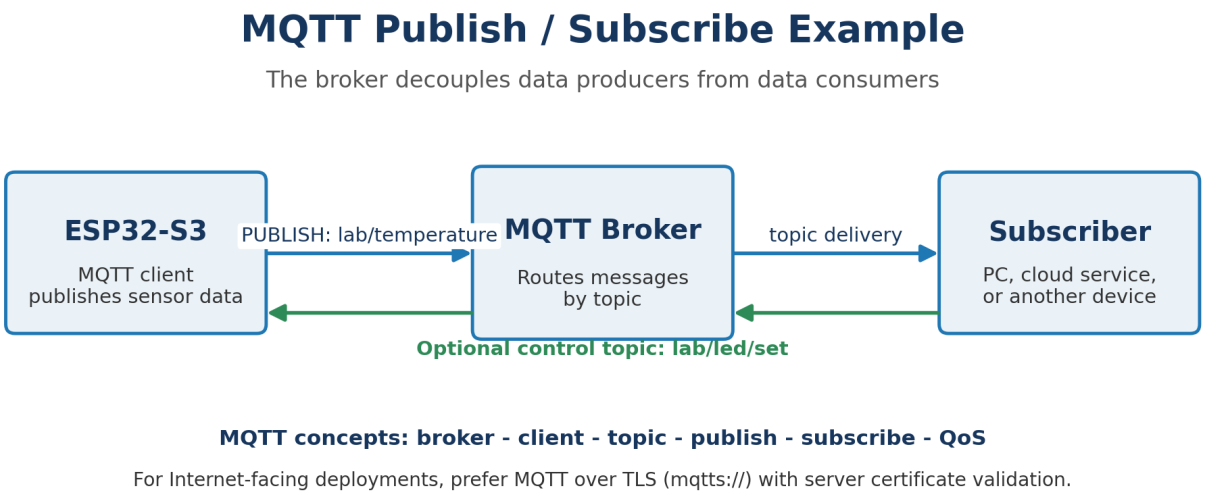


Figure 12. MQTT publish/subscribe flow.

```
#include "mqtt_client.h"

static void mqtt_start(void)
{
esp_mqtt_client_config_t mqtt_cfg = {
.broker.address.uri = "mqtt://192.168.1.50",
};

esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_cfg);
ESP_ERROR_CHECK(esp_mqtt_client_start(client));

esp_mqtt_client_publish(client,
"lab/esp32s3/temperature",
"23.7",
0, 1, 0);
}
```

For production systems, also configure reconnection behavior, authentication, TLS where required, message quality of service (QoS), topic naming, and event handling. For laboratory work, students should first understand broker, publisher, subscriber, topic, payload, QoS, and retained messages.

12 microSD Storage Example

Local storage is useful when the network is unavailable, when raw data must be preserved, or when a sensor node needs a local audit trail. ESP-IDF supports FAT filesystems on SD cards using SDMMC or SDSPI. A simple file example shows mount, open, write, close, and unmount operations.

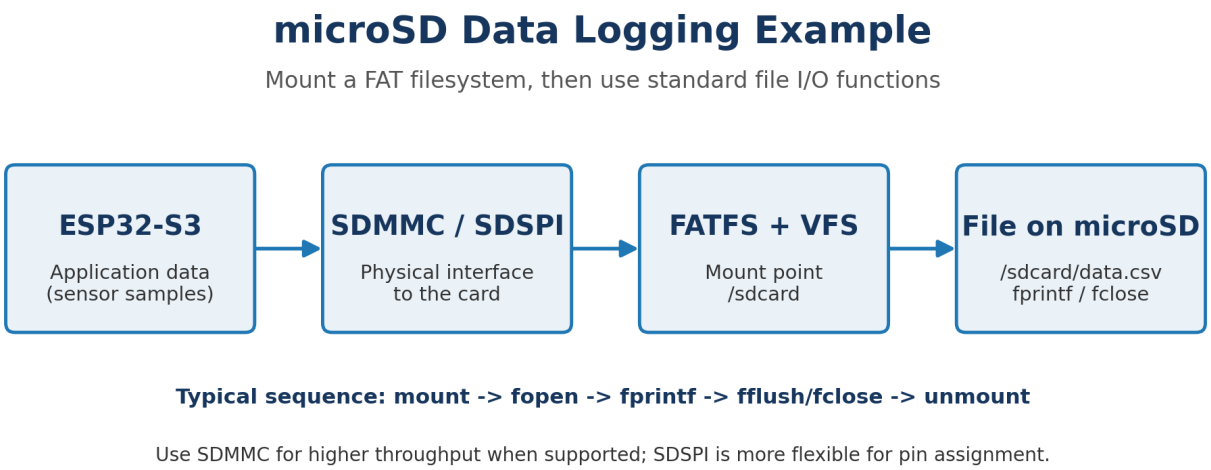


Figure 13. Typical microSD logging flow in ESP-IDF.

```
FILE *f = fopen("/sdcard/data.csv", "a");
if (f == NULL) {
ESP_LOGE("SD", "Failed to open file");
return;
}

fprintf(f, "timestamp,temperature,accel_x\n");
fprintf(f, "%ld,%.2f,%.3f\n", time(NULL), 23.7, 0.015);
fclose(f);
```

Data integrity: Always close or flush files before power is removed. For frequent logging, define a buffering strategy rather than opening and closing the card for every sample.

13 Connection Diagrams for Common Modules

Software examples become much easier to understand when students can see the corresponding physical wiring. The diagrams below use a simplified ESP32-S3 development board. Exact header positions vary by development board, so students must verify the board pinout before wiring.

13.1 I2C sensor: BME280 / MPU6050

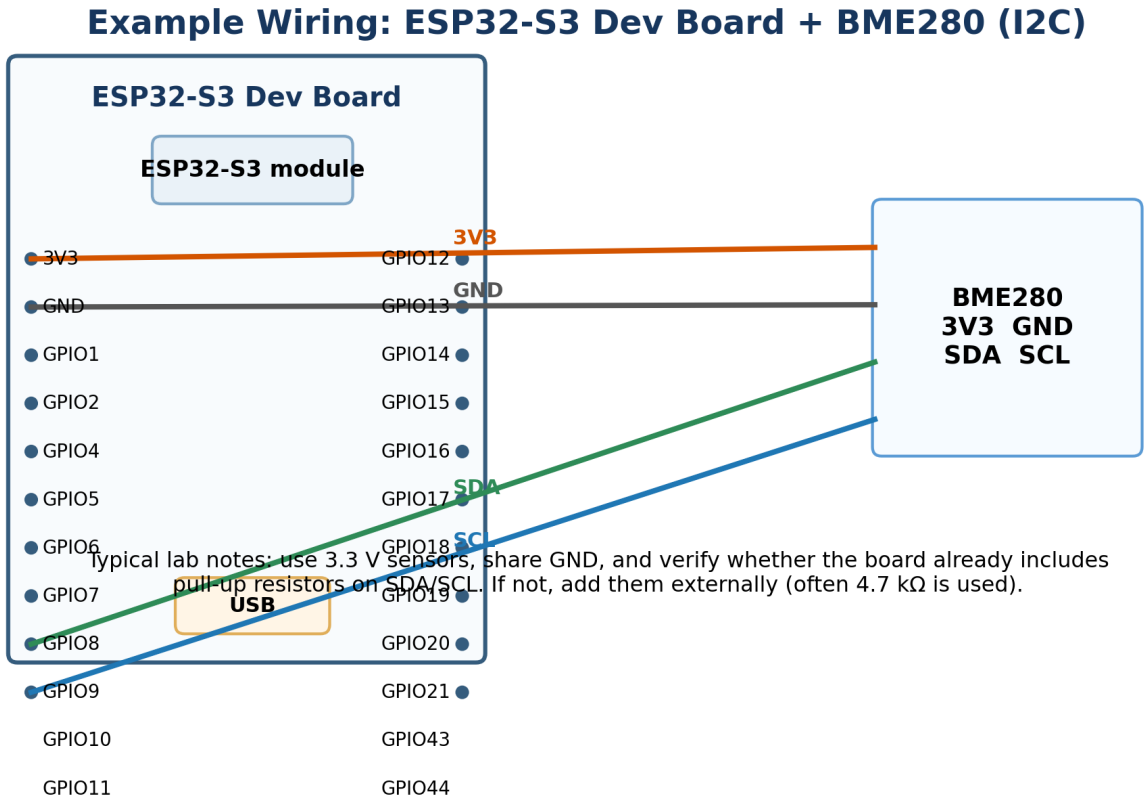


Figure 14. Typical I2C wiring between an ESP32-S3 development board and an I2C sensor.

- Connect 3.3 V and common ground.
- Connect SDA to the GPIO configured as sda_io_num.
- Connect SCL to the GPIO configured as scl_io_num.
- Verify whether the breakout board already includes pull-up resistors.

13.2 UART sensor: GPS module

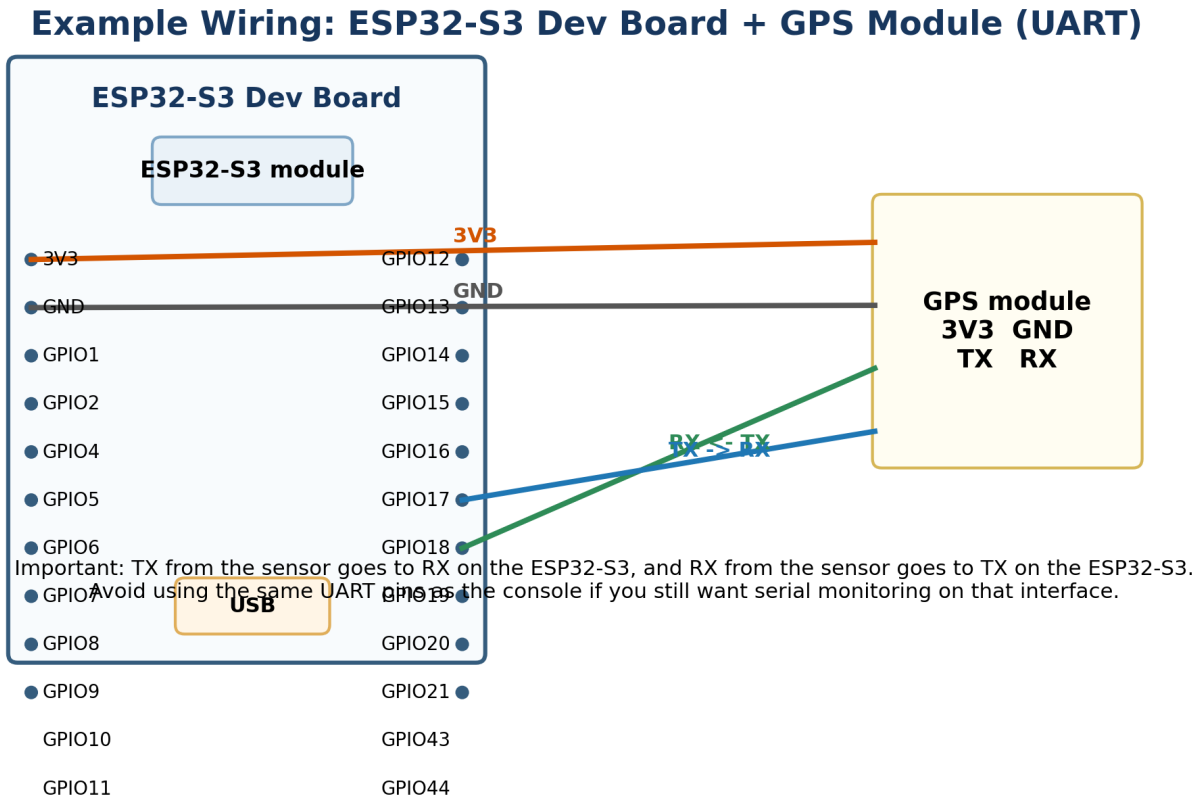


Figure 15. Example UART wiring between the ESP32-S3 and a GPS module.

- Sensor TX connects to ESP32-S3 RX.
- Sensor RX connects to ESP32-S3 TX.
- Use the correct supply voltage and logic levels.
- Keep the application UART separate from the console when possible.

13.3 SPI storage: microSD module

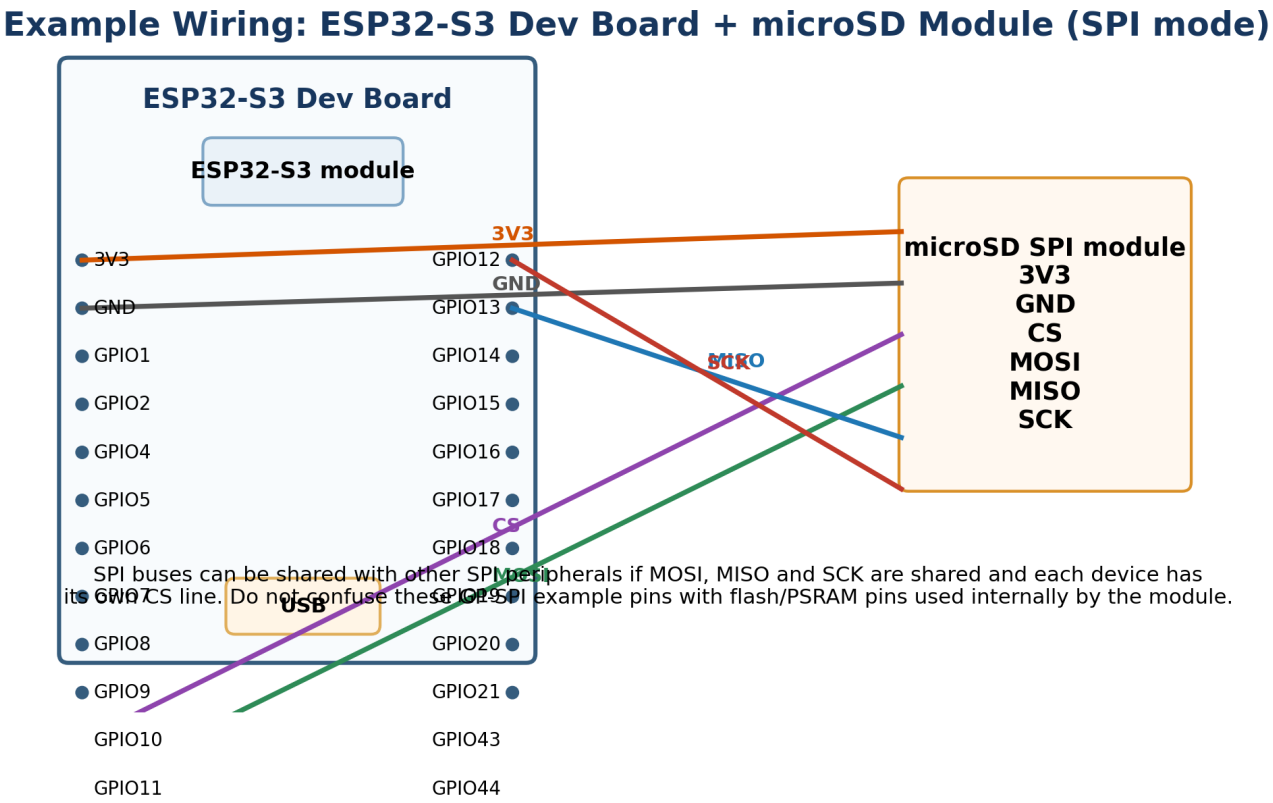


Figure 16. Example SPI wiring between the ESP32-S3 and a microSD module.

- MOSI, MISO, and SCK may be shared by multiple SPI devices.
- Each SPI device needs its own chip-select signal.
- Do not use flash/PSRAM-connected GPIOs for general-purpose SPI devices.

14 Complete Hello World Build, Flash, and Monitor Example

The hello_world example is the best first validation exercise because it tests the complete toolchain: project selection, target selection, compilation, firmware flashing, and runtime monitoring.

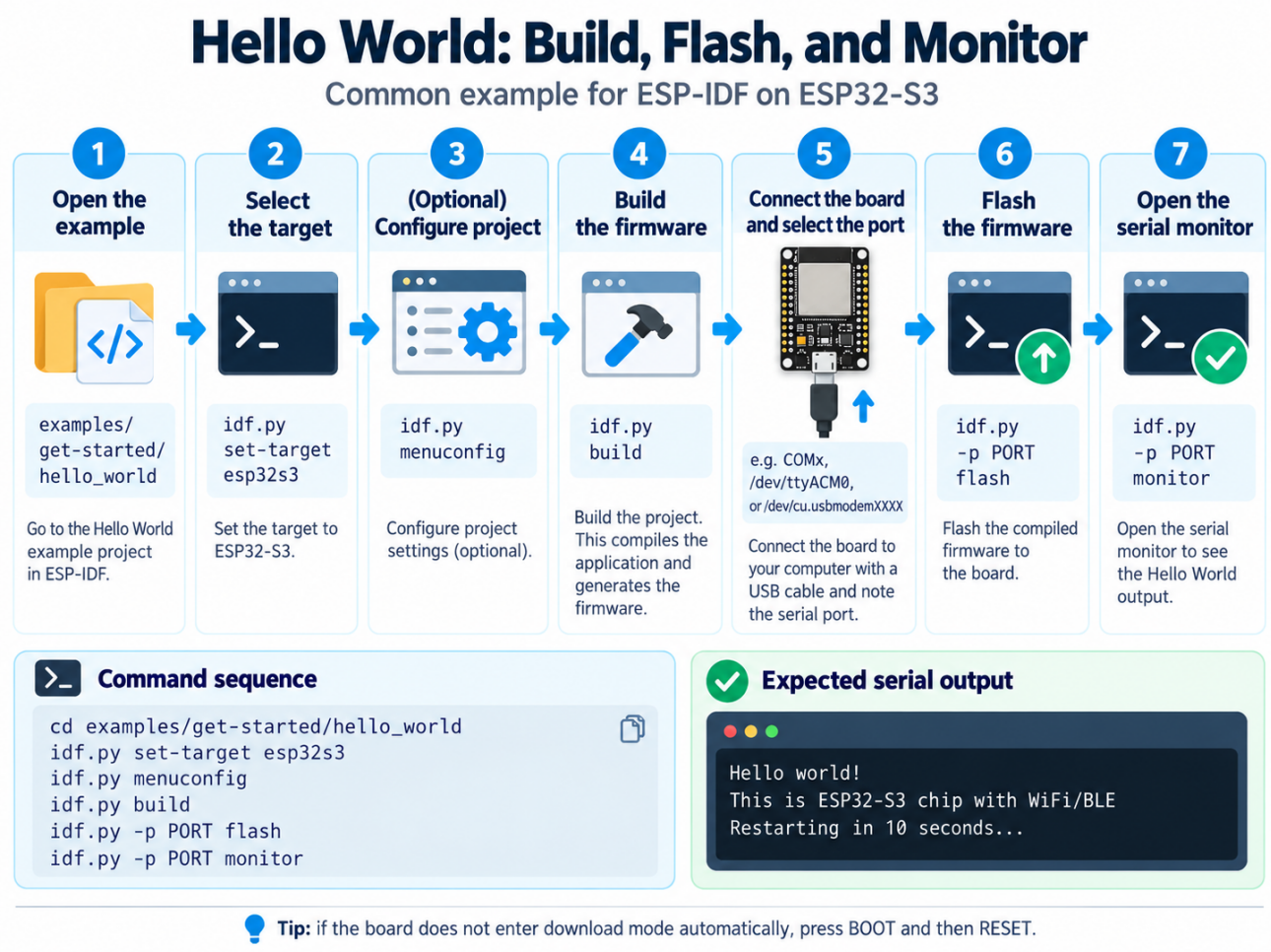


Figure 17. Complete Hello World workflow for ESP32-S3.

14.1 Step 1 - Open the example

```
# Windows (ESP-IDF PowerShell)
cd %IDF_PATH%\examples\get-started\hello_world

# Linux/macOS
cd $IDF_PATH/examples/get-started/hello_world
```

14.2 Step 2 - Select the target

```
idf.py set-target esp32s3
```

This command selects the ESP32-S3 architecture and regenerates target-specific project configuration as needed. Run it once when starting from a generic example or when changing target families.

14.3 Step 3 - Optional project configuration

```
idf.py menuconfig
```

menuconfig exposes project options through a text-based configuration interface. Students do not need to modify settings for the basic Hello World example, but they should learn where component, bootloader, flash, logging, and serial settings are configured.

14.4 Step 4 - Build the firmware

```
idf.py build
```

The build step runs CMake and Ninja, compiles ESP-IDF components and application sources, links the firmware, and generates binary images. A successful build does not mean the board has been programmed yet.

14.5 Step 5 - Identify the serial port

```
# Windows: example
COM3

# Linux: common examples
/dev/ttyACM0
/dev/ttyUSB0

# macOS: common example
/dev/cu.usbmodemXXXX
```

14.6 Step 6 - Flash the firmware

```
# Windows
idf.py -p COM3 flash

# Linux
idf.py -p /dev/ttyACM0 flash

# macOS
idf.py -p /dev/cu.usbmodemXXXX flash
```

During flashing, esptool identifies the chip, synchronizes with the bootloader, writes the binary images to flash memory, and verifies the operation. If automatic download mode fails, some boards require holding BOOT while pressing RESET/EN.

14.7 Step 7 - Monitor the program

```
idf.py -p PORT monitor

# Or flash and monitor in one command:
idf.py -p PORT flash monitor
```

The monitor displays both bootloader/system logs and application output. This is one of the most important debugging tools in ESP-IDF.

14.8 Minimal custom Hello World

```
#include <stdio.h>
#include "esp_log.h"

static const char *TAG = "HELLO";

void app_main(void)
{
printf("Hello World!\n");
ESP_LOGI(TAG, "ESP-IDF is running on ESP32-S3");
}
```

15 Troubleshooting and Best Practices

Problem	Likely cause	Recommended action
Board not detected	Charge-only cable or missing driver	Use a data cable, check USB driver and another port
Permission denied on Linux	User cannot access /dev/tty*	Add user to the correct serial group and log in again
idf.py not found	ESP-IDF environment not loaded	Open an ESP-IDF terminal or run export.sh
Build fails after target/config change	Stale build directory	Run idf.py fullclean and rebuild
Flash fails	Wrong port, board not in download mode, cable problem	Verify the port, reset/BOOT sequence, and cable
Monitor shows garbage	Wrong interface/baud or another device using the UART	Verify console configuration and port
I2C device not found	Wrong address, wrong pins, missing pull-ups	Run an address scan and check SDA/SCL wiring
ADC values unstable	Noise, poor grounding, wrong attenuation	Improve grounding, average samples, configure ADC correctly

Problem	Likely cause	Recommended action
Wi-Fi does not connect	SSID/password/security mismatch	Check event logs and access-point settings
MQTT cannot connect	Wrong broker address/port or network path	Test IP connectivity first, then MQTT configuration

Development habit: Change one subsystem at a time. First prove the board and toolchain with Hello World, then test a peripheral by itself, then add networking, and only after that combine the pieces into a full application.

16 Recommended References

- ESP-IDF Get Started - ESP32-S3
- ESP32-S3 GPIO and RTC GPIO
- ESP32-S3 I2C Driver
- ESP32-S3 SPI Master Driver
- ESP32-S3 Datasheet
- ESP-IDF GitHub Repository
- ESP-IDF Examples
- UCM IoT LAB1 - ESP-IDF Introduction

End of guide: Students should now be able to install ESP-IDF, understand the ESP32-S3 peripheral model, choose and remap GPIOs, connect common sensor modules, use ADC/UART/I2C/SPI, connect to Wi-Fi, create TCP and MQTT clients, write data to microSD, and validate a complete firmware build/flash/monitor workflow.